Module 2: Getting to Know Amazon Bedrock

Research Findings — Current State, February 2026

MODULE 2

AMAZON BEDROCK

Amazon Bedrock Fundamentals

Amazon Bedrock is a **fully managed, serverless AI service** that gives teams immediate access to foundation models without the burden of infrastructure management. Its architecture is built for enterprise-grade reliability, security, and scale.

Serverless & Managed

No infrastructure to provision or maintain. AWS handles scaling, availability, and patching automatically.

Multi-Tenant Isolation

Strong data and compute isolation guarantees across tenants, with no cross-customer data exposure.

Regional by Design

Models and data remain within your selected AWS Region, supporting data residency and compliance requirements.

API-First

All capabilities are accessible via APIs and SDKs — fully automatable and integrable with existing pipelines.

Foundation Models

100+ models from Amazon, Anthropic, Meta, Mistral, OpenAI, and more — accessed through a **unified API surface**. Model weights are never exposed to your account for hosted FMs.

Model Access & Availability

Access is requested per model in the Bedrock console and granted at the account/Region level. Some models are immediately available; others require provider approval. Regional availability and service tier options vary by model.

API Patterns Deep Dive

Amazon Bedrock exposes **four distinct API patterns** in 2026, each optimized for different use cases — from OpenAI-compatible migration paths to native low-level model invocation.

1

Responses API

Recommended for new projects. OpenAI Responses API–compatible schema via `bedrock-mantle.{region}.api.aws`. Best for stateful conversations, agentic workflows, built-in tool use, and server-side tool execution. Supports multimodal inputs and streaming.

2

Chat Completions API

Best for OpenAI Chat Completions migrations. Available on both the OpenAI-compatible mantle endpoint and native runtime. Works with existing OpenAI-compatible SDKs. Ideal for stateless or application-managed multi-turn chat and text-focused tasks.

3

Converse API

Native Bedrock interface. Unified schema across Amazon, Anthropic, and other models via `bedrock-runtime.{region}.amazonaws.com`. First-class support for system prompts, tools, guardrails, and streaming via `ConverseStream`.

4

Invoke API

Maximum control. Direct model invocation with `InvokeModel` (synchronous) or `InvokeModelWithResponseStream` (streaming). Best for single-turn transactions, large custom payloads, and model-specific request schemas.

Python SDK (boto3) Basics

Installation & Authentication

Install the SDK with `pip install boto3 botocore`. Bedrock uses standard AWS credential mechanisms — IAM roles, named profiles, or environment variables. Ensure your IAM policy includes actions such as `bedrock:InvokeModel` and `bedrock:Converse`.

Two Clients, Two Planes

- **bedrock** client — control plane: model catalog, access management, fine-tuning jobs
- **bedrock-runtime** client — inference plane: Invoke, Converse, Chat Completions

Client Setup

```
import boto3
region = "us-east-1"

# Control plane
bedrock_client = boto3.client(
    "bedrock",
    region_name=region
)

# Inference plane
bedrock_runtime = boto3.client(
    "bedrock-runtime",
    region_name=region
)
```

Converse API Example

```
response = bedrock_runtime.converse(
  modelId="anthropic.claude-3-5-sonnet-v2",
  messages=[{
    "role": "user",
    "content": [{"text": "Hello!"}]
  }],
  inferenceConfig={
    "maxTokens": 512,
    "temperature": 0.5
  }
)
```

InvokeModel Example

```
import json
response = bedrock_runtime.invoke_model(
  modelId="anthropic.claude-3-5-sonnet-v2",
  body=json.dumps({
    "messages": [{
      "role": "user",
      "content": "Your prompt here"
    }],
    "max_tokens": 300,
    "temperature": 0.7
  })
)
```

Note: Request schema differs by model family (Anthropic vs. Nova vs. GPT-OSS). Always consult model-specific docs.

Model Selection Criteria

Choosing the right model in Bedrock requires evaluating **seven key dimensions**. There is no universal best model — the optimal choice is always workload-specific.

Modality

Choose text-only or multimodal models (text + image + document + video). Match input types — text, images, documents, audio — to expected outputs.

Model Class & Size

Light (7B–20B): fast and cheap for high-volume tasks. **General**: balanced quality vs. latency. **Flagship/Frontier**: best reasoning at higher cost. Use benchmarks, not just parameter counts.

Context Window

Ranges from 4K–8K (small models) to 32K–128K (modern LLMs) to 200K+ tokens (Claude variants). Larger windows increase cost but enable long-document RAG and complex histories.

Inference Latency

Real-time apps need faster models and higher service tiers. Batch/offline workloads can tolerate higher latency for better quality. Service tier (Priority / Standard / Flex) directly affects throttling risk.

Pricing

Billed separately for input and output tokens; output is typically more expensive. GPT-OSS models are significantly cheaper than flagship proprietary models. Always check the Bedrock pricing page for current Region-specific values.

Fine-Tuning & Custom Models

Selected Amazon Titan/Nova and partner models support fine-tuning or continued pretraining. Use **Custom Model Import** or SageMaker JumpStart to bring your own models while retaining Bedrock tooling.

Special Capabilities

Tool use/function calling, advanced reasoning (GPT-OSS, DeepSeek, Claude frontier), code generation (Qwen Coder, Nova), and multilingual support vary by model family.

Pricing Models (2026)

Bedrock offers **four complementary pricing levers** that, when used together, can dramatically reduce total inference cost while maintaining performance for the right workloads.

On-Demand

Pay per input/output token with no minimum commitment. Service tiers — **Priority**, **Standard**, and **Flex** — apply per model.

Example: Claude 3.5 Sonnet ≈ **$3/1M input** · **$15/1M output** tokens (Standard, one Region, subject to change).

Reserved Capacity

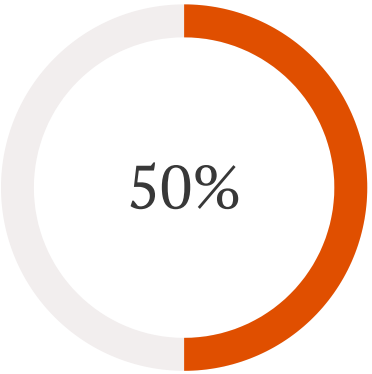
1- or 3-month commitments for specific models and tiers. Provides **guaranteed throughput** and tighter SLAs. Ideal when sustained usage is a large fraction of peak demand.

Batch Processing

Asynchronous jobs with S3 inputs/outputs processed over a multi-hour window. Delivers up to **~50% savings** vs. on-demand for eligible models. Best for content moderation, analytics, training data generation, and offline evaluations.

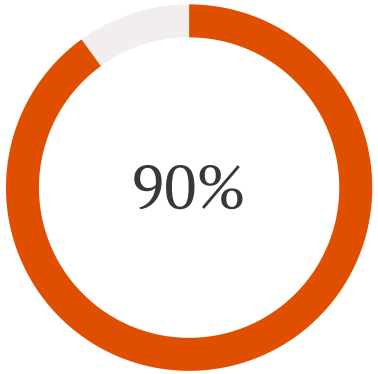
Prompt Caching

Reuse long prompt prefixes across many requests. Cache reads receive up to a **~90% discount** on input tokens. Currently supported on select Anthropic Claude and Amazon models, with coverage expanding.



Batch Savings

Approximate token cost reduction vs. on-demand for eligible batch inference workloads



Cache Discount

Potential discount on cached input tokens using prompt caching on supported models

Error Handling and Rate Limits

Building production-grade Bedrock applications requires anticipating and gracefully handling the four most common error types — and understanding how service tiers affect your quota headroom.

429 ThrottlingException

Cause: Exceeded requests-per-second or tokens-per-second limits.
Solution: Implement exponential backoff with jitter, reduce concurrency, or request a quota increase. Upgrade to Priority or Reserved tiers for higher limits and lower throttling risk.

503 ServiceUnavailableException

Cause: Temporary capacity or service-side issues.
Solution: Retry with backoff. For critical workloads, architect for cross-Region inference failover to maintain availability during regional disruptions.

400 ValidationException

Cause: Invalid parameters or malformed payload structure for the target model or API.
Solution: Validate request schemas carefully. Consult model-specific docs for max token limits, required fields, and payload structure differences across model families.

403 AccessDeniedException

Cause: Missing IAM permissions or model access not granted for the account/Region.
Solution: Update IAM policies to include required Bedrock actions and confirm model access is enabled in the Bedrock console for the correct account and Region.

Rate Limit Tiers at a Glance

1	2	3
Flex	**Standard**	**Priority / Reserved**
Lowest RPM/TPM quotas. Best for experimental and non-urgent workloads.	Moderate quotas and balanced throttling. Default choice for most production workloads.	Highest quotas, lowest throttling risk. Designed for latency-sensitive, mission-critical traffic.

Monitoring and Observability

CloudWatch Metrics (Automatic)

Bedrock automatically emits the following metrics for every model invocation — no additional configuration required to start collecting them:

- ### Invocations

  Total number of requests sent to a model endpoint.

- ### InvocationLatency

  End-to-end latency per request — critical for SLA monitoring.

- ### InvocationThrottles

  Count of throttled requests; a key signal for tier upgrade decisions.

- ### InputTokenCount / OutputTokenCount

  Token volumes for cost attribution and budgeting.

- ### 4xx / 5xx Error Counts

  Client errors (validation, auth) vs. server errors (capacity, service issues).

Model Invocation Logging

Beyond metrics, enable **Model Invocation Logging** to capture rich per-request data. Logs can be routed to **CloudWatch Logs**, **Amazon S3**, or both.

Log records can include:

- Full request and response payloads
- Input and output token counts
- Timestamps and request metadata
- Model ID and invocation parameters

Use invocation logs for **debugging**, **safety reviews**, **quality evaluation**, and **cost analysis**. Combine with CloudWatch dashboards for end-to-end visibility from day one.